

DHRUVA CDM

Conjunction Data Management System

Engineering & Technical Documentation

Project	Dhruva CDM — Orbital Conjunction Data Management
Stack	Python · FastAPI · React · Next.js · WebSocket · physics.py
Classification	Engineering Deep Dive — Internal Technical Reference
Version	1.0.0 — Initial Release
Repository	github.com/jeevankumar-m/project-dhruva
Demo Video	https://youtu.be/fs9eEaVNgIA

Abstract—Dhruva CDM is a full-stack orbital Conjunction Data Management (CDM) system designed for real-time space situational awareness (SSA), autonomous collision-avoidance maneuver scheduling, and mission-quality analytics. The system integrates a physics-accurate propagation engine—implementing two-body gravitational dynamics augmented with J2 oblateness perturbations and fourth-order Runge–Kutta (RK4) numerical integration—with a service-oriented FastAPI backend and a modern React operator interface. Conjunction assessment is performed via a dual-mode candidate filtering pipeline using either a 3D spatial grid pre-filter or a KD-tree proximity query, followed by Time-of-Closest-Approach (TCA) evaluation and risk labelling across SAFE/WARNING/CRITICAL thresholds. Operational updates are streamed to the frontend via WebSocket, providing sub-second latency for warning indicators. This document presents the system architecture, physics foundations, API surface, data flow, and design rationale in the style of an IEEE research contribution.

I. Introduction

The proliferation of objects in Earth orbit—spanning operational satellites, defunct spacecraft, upper-stage rocket bodies, and fragmentation debris—has elevated conjunction assessment from a specialist concern to a routine operational necessity. A Conjunction Data Management (CDM) system must perform three coordinated functions: (1) continuously propagate the state vectors of all tracked objects, (2) identify pairs whose predicted trajectories converge within actionable proximity thresholds, and (3) schedule or recommend avoidance maneuvers that preserve mission objectives while minimising delta-v expenditure and fuel consumption.

Dhruva CDM was developed to address these requirements in a single integrated platform. The project originated as a compact 2D orbital simulation and evolved, through a series of deliberate architectural expansions, into a production-grade service capable of ingesting real TLE data, simulating hundreds of debris objects, and delivering live risk assessments to an operator dashboard. This document provides a complete engineering reference: physics model, numerical methods, system architecture, API contract, data-flow specification, and operational guidelines.

II. System Evolution and Design Philosophy

II.A Genesis: simulator.py

The foundational proof-of-concept, `simulator.py`, placed a single satellite and a single debris object in a 2D Matplotlib scene. Both objects were propagated with RK4 integration; closest-approach prediction was performed over a finite look-ahead horizon; and if the predicted miss distance dropped below a configurable safety threshold, a small Δv impulse was applied in the instantaneous velocity direction. This minimal kernel validated three architectural invariants that survive into the production system:

- **Physics-first propagation:** object states are always derived from numerical integration, never interpolated from pre-computed tables.
- **Predictive conjunction assessment:** risk decisions are based on future state projections, not current position.
- **Closed-loop autonomy:** a maneuver planner consumes conjunction outputs and schedules corrective burns without operator intervention.

Figures 1–3 show three successive snapshots from the `simulator.py` proof-of-concept, illustrating the simulation evolving from initial trajectory separation through conjunction approach to post-evasion recovery.

II.B Architectural Growth

After validating the physics loop, the monolithic simulator was decomposed into a service-oriented architecture. The

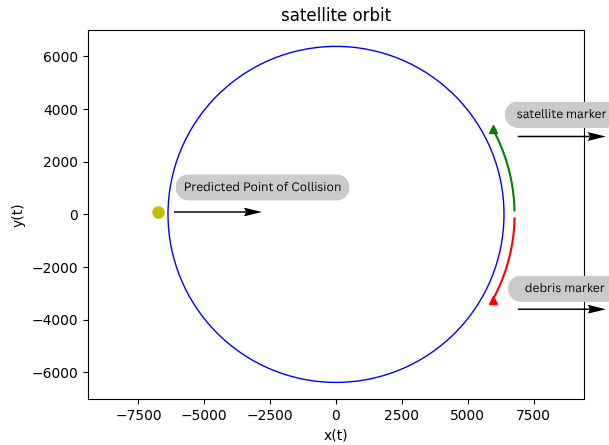


Figure 1: Genesis Stage 1 — Initial 2D simulation state. The satellite (green) and debris (red) propagate on separate tracks; the predicted point of collision (yellow marker) is identified on the look-ahead horizon.

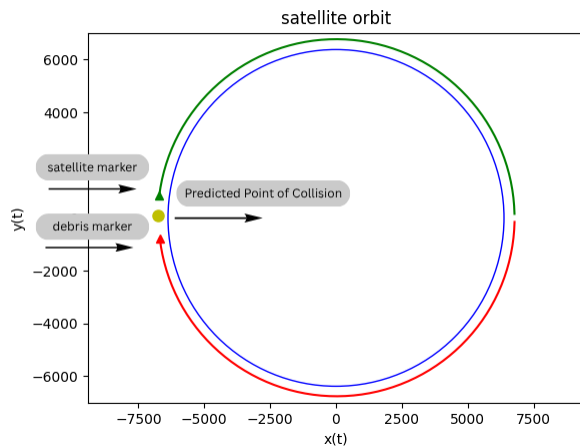


Figure 2: Genesis Stage 2 — Conjunction approach. Orbits converge and the predicted collision point migrates toward the satellite position, triggering the threshold check that will schedule the evasion burn.

backend became responsible for state propagation, conjunction assessment, maneuver scheduling, blackout validation, and metrics accumulation. The frontend consumed these services to render map views, 3D orbit visualisations, warning overlays, analytics dashboards, and operator workflow panels. A critical inflection point was the transition from REST polling to WebSocket streaming for live update delivery—discussed in detail in Section 3.

III. System Architecture

III.A High-Level Overview

The Dhruva CDM system is structured around four primary layers: the physics engine, the simulation state manager, the FastAPI service layer, and the React operator frontend. Figure 4 illustrates the component topology and communication pathways.

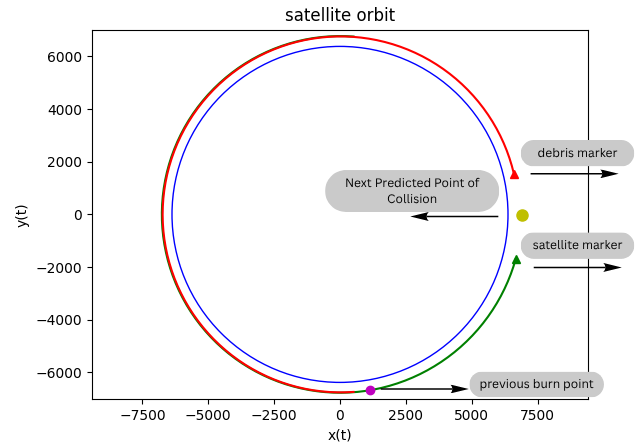


Figure 3: Genesis Stage 3 — Post-evasion state. The Δv impulse has displaced the satellite onto a safe trajectory; the previous burn point (magenta) and the next predicted conjunction (yellow) are both visible, confirming closed-loop autonomy.

III.B Module Responsibilities

Each module in the backend has a clearly bounded responsibility, as summarised in Table 1.

III.C Live Update Architecture — Polling vs. WebSocket Streaming

A polling model has the client repeatedly requesting snapshots at fixed intervals. While simple to implement, polling introduces several operational liabilities: missed fast transitions between poll intervals, stale warning states visible to operators during high-activity periods, and unnecessary request overhead under heavier compute loads.

Dhruva CDM adopts a *server-driven WebSocket streaming model*. In `server.py`, the simulation advances via `state.stream.tick()` and immediately publishes a fresh snapshot through `ws.send_json(state.snapshot())`. This design yields three measurable operational benefits:

- Lower end-to-end latency for warning state updates—the UI reflects risk changes within a single tick rather than the next poll interval.
- Smoother visual feedback for continuously animated elements such as orbital tracks and Bullseye risk indicators.
- Elimination of synchronisation artifacts between the compute cadence and the UI refresh cadence.

IV. Physics Engine — Mathematical Foundations

The physics engine (`physics.py`) implements the astrodynamical models required for accurate short- and medium-term orbit propagation. All computations are performed in Earth-Centred Inertial (ECI) Cartesian coordinates with distances in kilometres and velocities in kilometres per second.

IV.A Physical Constants

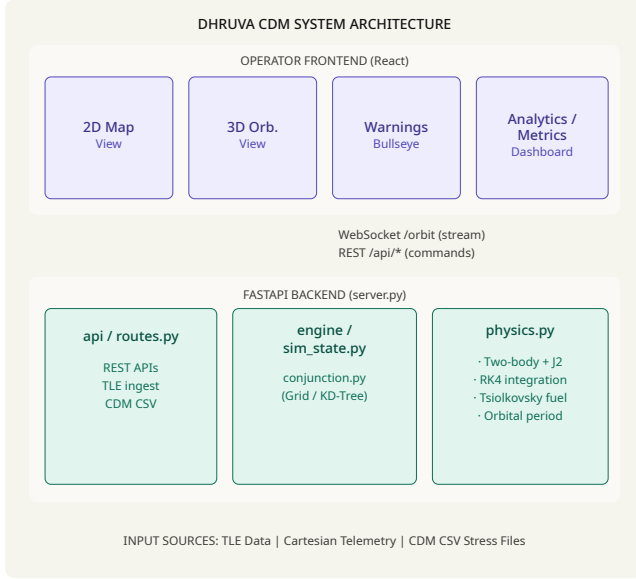


Figure 4: Dhruva CDM System Architecture — Component Topology and Communication Pathways.

IV.B Two-Body Gravitational Acceleration

The central gravitational acceleration for a body at position vector $\vec{r} = [x, y, z]^T$ with magnitude $|\vec{r}|$ is:

$$\vec{a}_{\text{grav}} = -\frac{\mu}{|\vec{r}|^3} \vec{r} \quad (1)$$

This provides the dominant acceleration term for all objects in Earth orbit and forms the baseline from which perturbations are added.

IV.C J2 Oblateness Perturbation

Earth's equatorial bulge produces a significant secular precession of the right ascension of the ascending node (RAAN) and argument of perigee, as well as short-period oscillations in all orbital elements. The first zonal harmonic, J_2 , captures this effect.

Define:

$$s = \frac{3}{2} \frac{J_2 \mu R_E^2}{|\vec{r}|^5} \quad (2)$$

Then the perturbation acceleration components are:

$$a_x = sx \left(\frac{5z^2}{|\vec{r}|^2} - 1 \right) \quad (3)$$

$$a_y = sy \left(\frac{5z^2}{|\vec{r}|^2} - 1 \right) \quad (4)$$

$$a_z = sz \left(\frac{5z^2}{|\vec{r}|^2} - 3 \right) \quad (5)$$

The total acceleration applied at every integration step is therefore:

$$\vec{a}_{\text{total}} = \vec{a}_{\text{grav}} + \vec{a}_{J_2} \quad (6)$$

Inclusion of J_2 is essential for realistic conjunction prediction because the secular drift in RAAN causes satellite and debris planes to diverge or converge on timescales of hours to days, profoundly affecting conjunction geometry.

Table 1: Module Responsibilities

Module	Responsibility
<code>server.py</code>	FastAPI app bootstrap, CORS config, <code>SimulationState</code> instantiation, REST route mounting, WebSocket <code>/orbit</code> endpoint.
<code>api/routes.py</code>	All REST API endpoint definitions: telemetry ingest, TLE ingest, maneuver scheduling, step simulation, snapshot retrieval, seed/reset, time-warp, CDM CSV loader.
<code>engine/sim_state.py</code>	Operational core: simulation time, satellite/debris state vectors, maneuver queues, conjunction warnings, black-out tracking, graveyard logic, objective metrics.
<code>engine/conjunction.py</code>	Conjunction candidate computation and risk labelling using grid or KD-tree pre-filter, then TCA evaluation and SAFE/WARNING/CRITICAL classification.
<code>physics.py</code>	Astrodynamics backbone: two-body + J2 dynamics, RK4 numerical integration, state propagation, Tsiolkovsky propellant model, orbital period estimation.
<code>dhruva-frontend</code>	React SPA: 2D map view, 3D orbit view, Bullseye risk display, analytics dashboard, operator maneuver workflow.

IV.D Fourth-Order Runge–Kutta Integration

State derivatives are integrated using the classical fourth-order Runge–Kutta (RK4) method. For state vector $\vec{S} = [x, y, z, v_x, v_y, v_z]^T$ and derivative function $f(\vec{S}) = [v_x, v_y, v_z, a_x, a_y, a_z]^T$:

$$\vec{k}_1 = f(\vec{S})$$

$$\vec{k}_2 = f\left(\vec{S} + \frac{1}{2}\Delta t \vec{k}_1\right)$$

$$\vec{k}_3 = f\left(\vec{S} + \frac{1}{2}\Delta t \vec{k}_2\right)$$

$$\vec{k}_4 = f\left(\vec{S} + \Delta t \vec{k}_3\right)$$

$$\vec{S}_{n+1} = \vec{S}_n + \frac{\Delta t}{6} (\vec{k}_1 + 2\vec{k}_2 + 2\vec{k}_3 + \vec{k}_4) \quad (7)$$

RK4 provides fourth-order local truncation error $\mathcal{O}(\Delta t^5)$ and global error $\mathcal{O}(\Delta t^4)$, giving an excellent balance of efficiency and accuracy for the step sizes used in operational simulation. The fixed-step implementation applies `rk4_step` iteratively over $\lfloor \text{duration}/\Delta t \rfloor$ steps.

IV.E Tsiolkovsky Propellant Model

Maneuver feasibility checks and fuel budgeting use the Tsiolkovsky rocket equation. The propellant mass consumed for a burn of magnitude $|\Delta \vec{v}|$ (in m/s) from a spacecraft of current

Table 2: Physical Constants in `physics.py`

Sym.	Value	Units	Description
μ	398600.4418	km^3s^{-2}	Earth gravitational parameter
J_2	1.08263×10^{-3}	—	Earth oblateness coefficient
R_E	6378.137	km	Earth equatorial radius
I_{sp}	300	s	Specific impulse (thrust)
g_0	9.80665	m s^{-2}	Standard gravitational acceleration

mass m_{current} is:

$$m_{\text{prop}} = m_{\text{current}} \left(1 - \exp \left(-\frac{|\Delta \vec{v}|}{I_{sp} g_0} \right) \right) \quad (8)$$

This model enables realistic pre-validation of commanded maneuvers against the satellite’s remaining fuel budget, preventing physically impossible burns from being scheduled.

IV.F Orbital Period Estimation

For operational displays and scheduling calculations, the Keplerian orbital period is estimated for near-circular orbits at altitude h :

$$a = R_E + h, \quad T = 2\pi \sqrt{\frac{a^3}{\mu}} \quad (9)$$

This relation is accurate to better than 0.1% for eccentricities below 0.01, covering the majority of operational spacecraft and debris populations addressed by Dhruva CDM.

V. Conjunction Assessment Pipeline

Conjunction assessment follows a two-stage pipeline: candidate pre-filtering in position space, followed by predictive TCA evaluation for each candidate pair. This architecture avoids the $\mathcal{O}(N^2)$ cost of evaluating all object pairs by first reducing the candidate set to a computationally tractable size.

V.A Stage 1: Candidate Pre-filtering

Spatial Grid Method. Debris objects are bucketed into a 3D spatial grid partitioned by configurable cell size. For each satellite, the cell containing the satellite and all directly adjacent cells are scanned. The grid method is selected for `stream.tick()`—the live frontend update path—because its bounded per-tick cost provides deterministic performance under variable debris counts, protecting UI responsiveness.

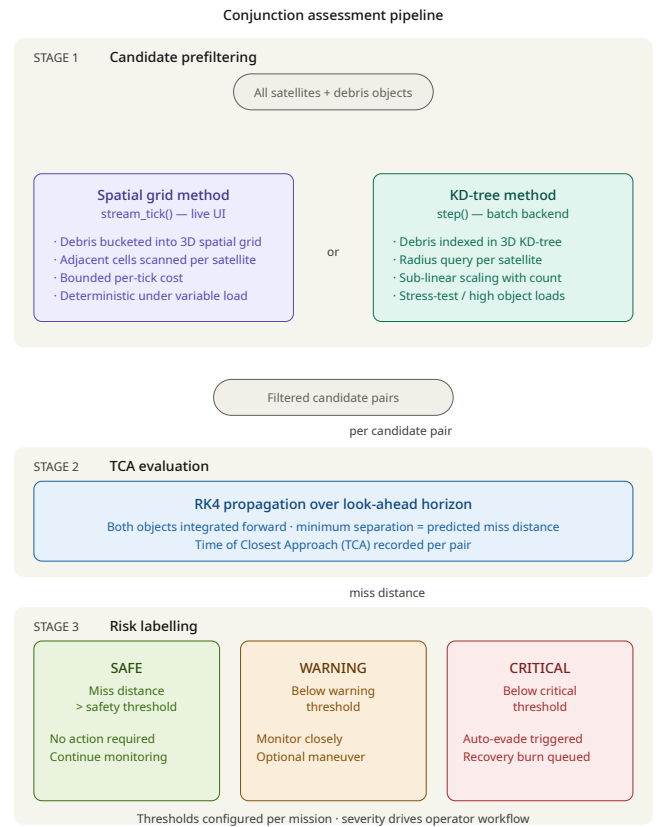
KD-Tree Method. Debris positions are indexed in a 3D KD-tree. For each satellite, all debris within a configurable search radius are returned via a single radius query. The KD-tree offers asymptotically better query performance for large, uniformly distributed populations and is selected for `step()`—the batch backend evaluation and stress-testing path.

V.B Stage 2: TCA Evaluation and Risk Labelling

For each candidate pair passing the pre-filter, both objects are propagated forward across a configurable look-ahead horizon using the RK4 integrator (Section IV). The minimum separation encountered during this propagation is recorded as the predicted miss distance. Risk is then assigned as shown in Table 3.

Table 3: Risk Classification Thresholds

Risk Level	Miss Distance	Operator Action
SAFE	> safety threshold	No action required.
WARNING	< warning threshold	Monitor; optional maneuver.
CRITICAL	< critical threshold	Auto-evade triggered.

**Figure 5:** Conjunction Assessment Pipeline — Dual-Mode Pre-filter and TCA Evaluation.

VI. API Reference

The Dhruva CDM backend exposes a REST API for command ingestion and a WebSocket endpoint for live state streaming. All REST endpoints are mounted under the `/api` prefix.

VI.A Telemetry and State Ingestion

VI.B Maneuver and Simulation Control

VI.C Data Retrieval

Table 4: Telemetry and State Ingestion Endpoints

Verb	Endpoint	Description
POST	/api/telemetry	Ingest Cartesian state vector $[x, y, z, v_x, v_y, v_z]$ in km and km-s ⁻¹ for a satellite or debris object.
POST	/api/telemetry/tle	Ingest a Two-Line Element set. Backend applies SGP4 propagation and ECI frame transformation before storing the resulting Cartesian state.
POST	/api/debug/load_cdm_csv	Load a CDM CSV file containing RTN relative state fields. Debris objects are anchored to in-simulation satellites for large-scale stress testing.

VII.D Snapshot Payload Schema

Each snapshot (whether returned by GET /api/snapshot or pushed over /orbit) contains the top-level fields listed in Table 7.

VII. Feature Catalogue

VII.A Maneuver Scheduling and Validation

Before any maneuver is accepted into the queue, the scheduler performs the following validation sequence:

- **Command latency check:** ensures the commanded burn time is sufficiently in the future to allow uplink delivery.
- **Thermal cooldown validation:** enforces a minimum inter-burn interval to protect thruster hardware.
- **Line-of-sight (LOS) constraint:** confirms the satellite will be visible to a ground station at the commanded burn epoch, enabling command upload and telemetry confirmation.
- **Fuel feasibility:** applies Eq. (8) to verify the commanded $|\Delta \vec{v}|$ is achievable with remaining propellant mass.

Autonomous evasion burns are generated by the conjunction engine for CRITICAL-risk events when no operator-scheduled maneuver is already queued. Each auto-evade is paired with a recovery burn scheduled to restore the original orbital plane after the conjunction window has passed.

VII.B Blackout Awareness

Ground station visibility is modelled as a binary LOS flag per satellite. When a satellite enters blackout, its blackout_start epoch is recorded and a recovery time estimate is derived from the orbital period. This information gates maneuver scheduling—burns cannot be commanded during blackout—and is surfaced to the operator dashboard to explain scheduling constraints.

VII.C Graveyard Maneuver Handling

Satellites whose remaining fuel mass drops below a configurable threshold are flagged as graveyard candidates. The system can automatically plan and schedule a disposal maneuver to move the satellite to a designated graveyard orbit.

Table 5: Maneuver and Simulation Control Endpoints

Verb	Endpoint	Description
POST	/api/maneuver/schedule	Schedule a maneuver burn. Validates command latency, thermal cooldown window, line-of-sight (LOS) constraint, and fuel feasibility before acceptance. Returns acceptance/rejection with reason.
POST	/api/sim/step	Advance the simulation by one configured time step using KD-tree conjunction mode. Suitable for batch evaluation workflows.
POST	/api/sim/seed	Seed or re-seed the simulation with an initial state, resetting all tracked objects, maneuver queues, and metrics.
POST	/api/sim/reset	Reset simulation to initial seeded conditions, clearing all accumulated history and warnings.
POST	/api/sim/timewarp	Set the simulation time-warp multiplier, controlling how many simulated seconds elapse per real second in stream mode.

Table 6: Data Retrieval Endpoints and WebSocket Stream

Verb	Endpoint	Description
GET	/api/snapshot	Pull current simulation snapshot: all state vectors, warnings, maneuver queue, blackout status, and objective metrics. Suitable for dashboard initialisation.
WS	/orbit	WebSocket stream endpoint. Server pushes fresh snapshots after each simulation tick. Recommended for all live visualisation and warning display.

The satellite status transitions through:

NOMINAL → GRAVEYARD_CANDIDATE → GRAVEYARD_MANEUVER_SCHEDULED → GRAVEYARD

providing clear operator visibility of the disposal lifecycle.

VII.D Objective Metrics

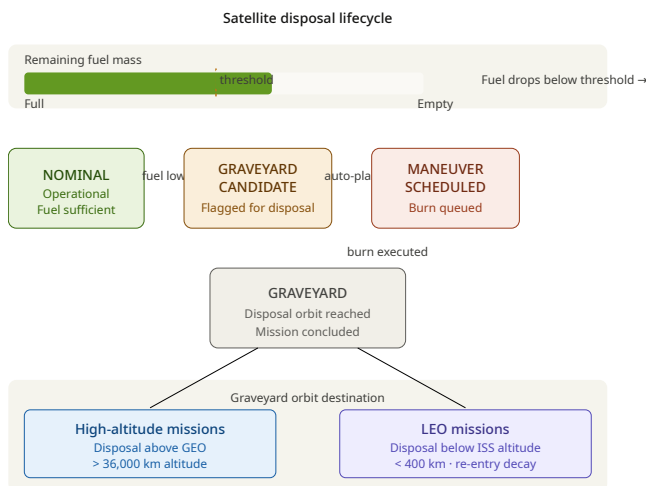
VIII. Data Flow — Input to Warning

The end-to-end data flow from object ingestion to operator warning display proceeds through five ordered stages (Figure 7).

1. **Ingestion:** Objects enter the simulation via direct Cartesian telemetry POST, TLE ingest with SGP4 + frame transformation, or CSV-derived RTN-to-ECI conversion anchored to in-simulation primaries.
2. **Propagation:** sim_state advances all objects with rk4_step using two-body + J2 dynamics. Satellites with

Table 7: Snapshot Payload Schema

Field	Type	Description
sim_time	float	Current simulation epoch in seconds since seed.
satellites	obj[]	Array of satellite state objects: id, position, velocity, fuel_mass, status.
debris	obj[]	Array of tracked debris state objects.
conjunctions	obj[]	Active conjunction warnings: sat_id, debris_id, tca_seconds, miss_distance_km, risk_level.
maneuvers	obj[]	Scheduled and completed maneuver records.
blackouts	obj[]	Per-satellite blackout status and estimated LOS recovery time.
metrics	object	Mission-quality metrics: uptime_pct, outage_sat_seconds, total_delta_v, avoidance_per_dv.

**Figure 6:** Graveyard Lifecycle — Satellite Disposal State Transitions.

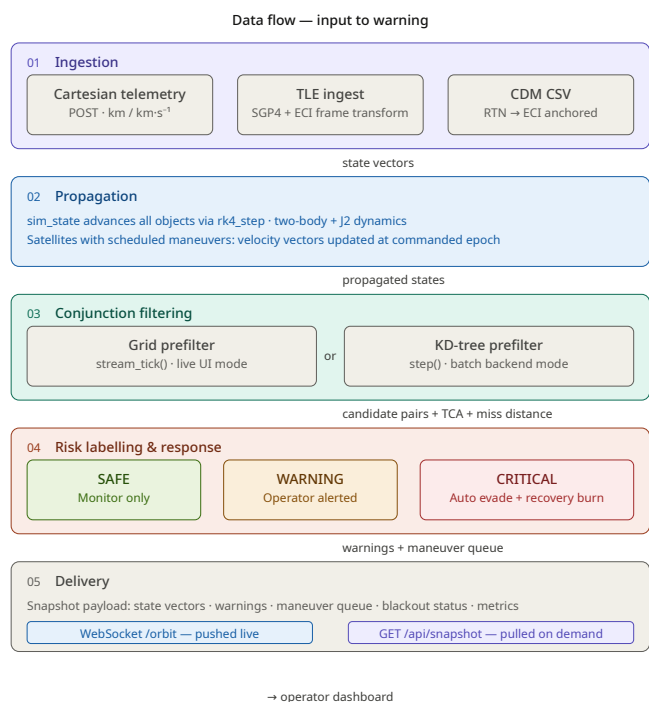
scheduled maneuvers have their velocity vectors updated at the commanded epoch.

3. **Conjunction Filtering:** Grid (stream mode) or KD-tree (step mode) pre-filter reduces the candidate pair set. Each candidate pair is propagated to TCA and miss distance is computed.
4. **Risk Labelling & Response:** SAFE/WARNING/CRITICAL thresholds are applied. CRITICAL events trigger autonomous evade/recovery burn generation. All warnings populate the conjunction warning list.
5. **Delivery:** State, warnings, maneuver queue, blackout status, and metrics are packaged into a snapshot payload and pushed over the WebSocket stream or returned via the snapshot REST endpoint.

IX. Operational Notes

Table 8: Mission-Quality Objective Metrics

Metric	Definition
uptime_pct	Percentage of simulation time during which all satellites are operational (not in blackout, not disposed).
outage_sat_seconds	Cumulative satellite-seconds lost to blackout or operational degradation.
total_delta_v	Total $ \Delta \vec{v} $ consumed across all scheduled and autonomous maneuvers (m/s).
avoidance_per_dv	Number of successful avoidance actions per m/s of $ \Delta \vec{v} $ expended — a composite efficiency score.

**Figure 7:** Five-Stage Data Flow: Ingestion to Operator Warning.

IX.A Backend Stress Testing

For high object-count backend evaluation, use KD-tree mode via step()-style API workflows. The KD-tree provides sub-linear scaling with debris count for radius queries, making it substantially faster than grid scanning when object counts exceed several hundred. CDM CSV files with large relative-state datasets can be loaded via POST /api/debug/load_cdm_csv to populate synthetic debris populations for performance benchmarking.

IX.B Operator Visualization

For smooth real-time operator visualization, always connect via the WebSocket /orbit endpoint and use the stream mode, which internally applies the grid pre-filter. The bounded per-tick cost of the grid method protects UI responsiveness under variable and unpredictable debris counts during live operations.

IX.C CDM CSV Frame Note

When loading CDM CSV stress data, the relative RTN state vectors are data-driven and accurately reflect the commanded relative geometry. However, the absolute ECI anchor frame is synthetic unless the primary satellite states in the simulation have been explicitly matched to the true primary states at the CDM epoch. Conjunction probabilities derived from such synthetic scenarios should be interpreted as performance benchmarks rather than real operational risk assessments.

IX.D Recommended Evaluation Workflow

1. Seed the simulation with representative satellite states (TLE-derived recommended).
2. Load debris population via CDM CSV or direct telemetry.
3. Connect frontend over WebSocket /orbit for live display.
4. Use POST /api/sim/step in parallel threads for batch conjunction assessment.
5. Monitor metrics via snapshot endpoint for mission-quality scoring.

X. Operator Dashboard — Frontend Visualizer

The Dhruva CDM frontend (dhruva-frontend) delivers situational awareness across six dedicated views: a 2D Mercator ground-track map, a 3D Cesium orbit tracker, the Conjunction Bullseye, the Fleet Fuel & Δv analytics page, the burn-log reports panel, the Graveyard Orbit Monitor, and the CDM Alerts feed. All views are driven by the live WebSocket stream and update within a single simulation tick.

X.A 2D Ground-Track Map (Mercator)

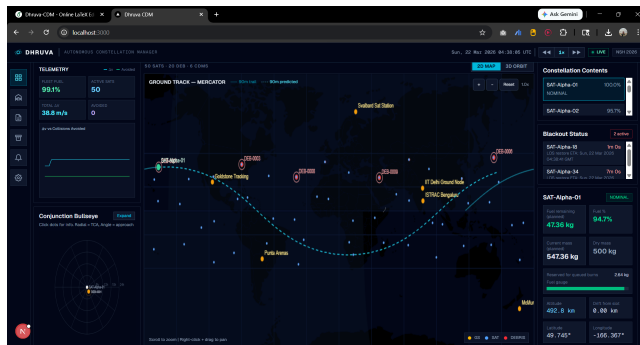


Figure 8: 2D Mercator ground-track view — 50 active satellites (cyan) and 20 debris objects (red) rendered live. The dashed cyan line shows the 90-minute predicted trajectory for the selected satellite (SAT-Alpha-01). Ground stations, blackout status, and the Conjunction Bullseye mini-panel are visible simultaneously.

X.B 3D Orbit Tracker (Cesium)

X.C Conjunction Bullseye — Extended View

X.D Fleet Fuel & Δv Analysis

X.E Reports — Burn Logs



Figure 9: 2D Mercator view with the Maneuver Timeline Gantt panel expanded at the bottom. Executed burns (green), cooldown windows (teal), and one failed command (red) for satellites A-07, A-09, A-01, and A-07R are visible. SAT-Alpha-01 shows a 3.69 km drift-from-slot, indicating a post-evasion recovery in progress.

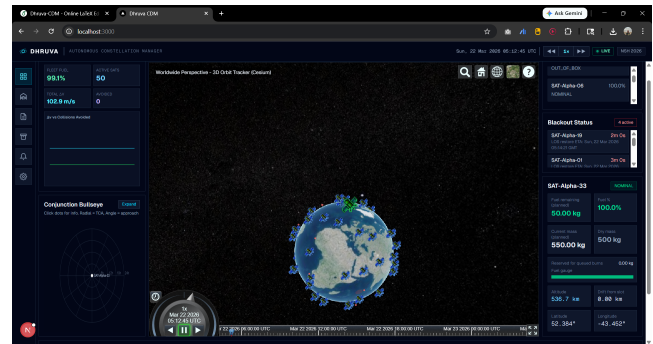


Figure 10: Worldwide 3D perspective in the Cesium globe renderer — the full 50-satellite constellation is visible as blue satellite icons distributed across low Earth orbit. Four active blackouts are flagged in the right panel. Fleet fuel holds at 99.1% with total Δv of 102.9 m/s consumed.

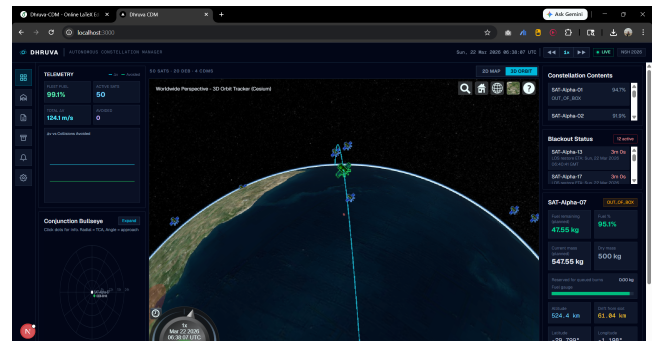


Figure 11: Close-up 3D horizon view over the Indian Ocean showing a cluster of satellites at orbital altitude ≈ 524 km. SAT-Alpha-07 is flagged OUT_OF_BOX with a 61.04 km drift from its nominal slot, confirming an active station-keeping recovery. Twelve simultaneous blackouts are tracked in the right panel.

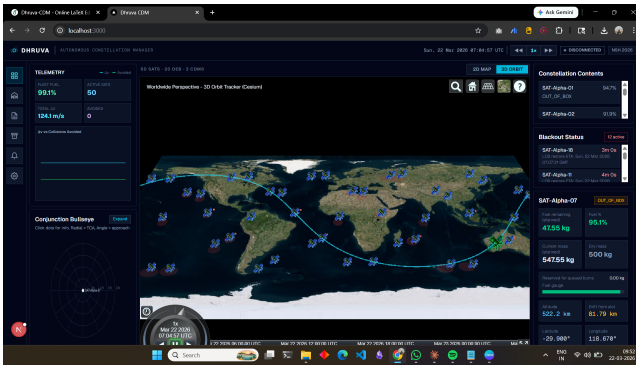


Figure 12: 3D Cesium view unfurled to a flat-Earth perspective, showing the global distribution of the constellation at 07:04 UTC. Debris objects (red) are spread across multiple orbital planes. SAT-Alpha-07 remains OUT_OF_BOX at 81.79 km drift; 12 blackouts remain active across the fleet.

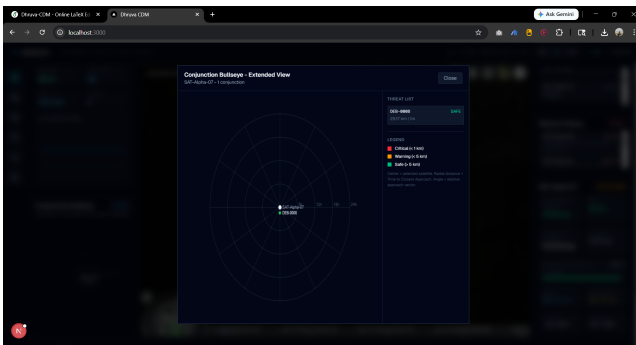


Figure 13: Conjunction Bullseye expanded modal for SAT-Alpha-07. DEB-0008 is plotted at 25.17 km miss distance with a 1-minute TCA, classified SAFE (green). Radial distance encodes time to closest approach; angle encodes the relative approach vector. The threat list on the right provides per-debris risk classification.

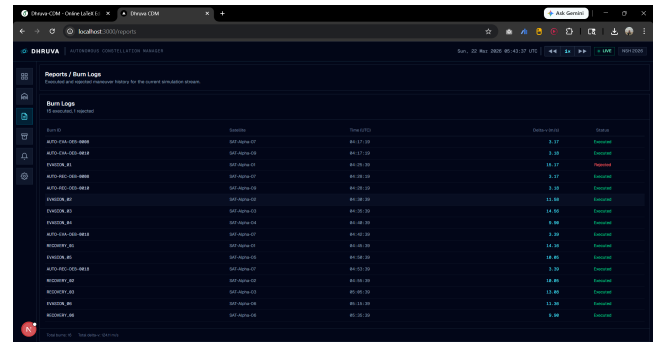


Figure 15: Reports / Burn Logs panel showing 16 total burns (15 executed, 1 rejected). Autonomous evasion burns (AUTO-EVA-*) and their paired recovery burns (AUTO-REC-*) are interleaved with operator-scheduled EVASION_ and RECOVERY_ sequences. EVASION_01 for SAT-Alpha-01 at 15.17 m/s was rejected, illustrating the LOS/cooldown validation in action.

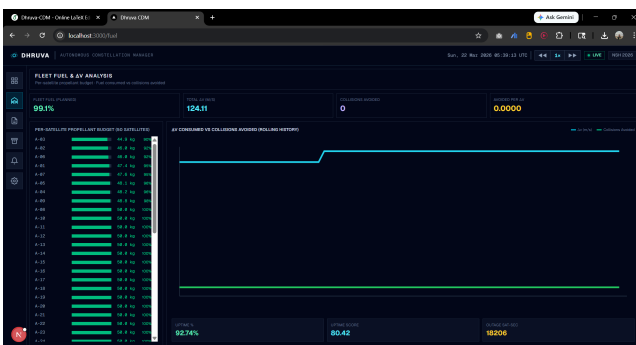


Figure 14: Fleet Fuel & Δv Analytics dashboard. The left panel shows per-satellite propellant budgets for all 50 satellites; A-03 leads consumption at 44.9 kg (90%). The right panel plots cumulative Δv (cyan, 124.11 m/s) against collisions avoided (green, rolling history). Fleet uptime is 92.74%, uptime score 80.42, and outage satellite-seconds total 18,206.

X.F Graveyard Orbit Monitor & CDM Alerts Feed

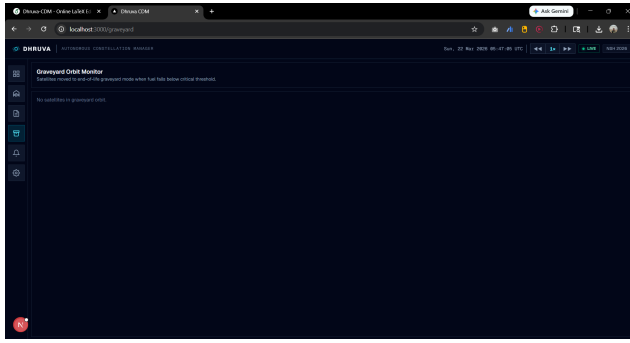


Figure 16: Graveyard Orbit Monitor at 05:47 UTC — no satellites have reached the fuel critical threshold, confirming that propellant budgeting and station-keeping logic has maintained the entire 50-satellite fleet above the end-of-life disposal trigger throughout the simulation run.

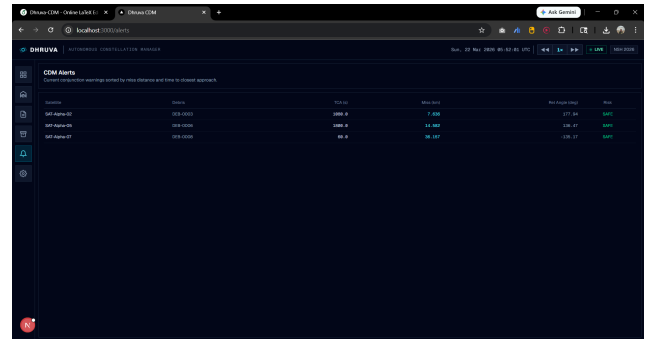


Figure 17: CDM Alerts feed at 05:52 UTC showing three active SAFE conjunction warnings: SAT-Alpha-02/DEB-0003 at 7.636 km (TCA 1080 s), SAT-Alpha-05/DEB-0006 at 14.582 km (TCA 1800 s), and SAT-Alpha-07/DEB-0008 at 36.157 km (TCA 60 s). Successful prior evasion maneuvers have preserved safe standoff distances across all tracked pairs.

XI. Conclusion and Future Work

XI.A Summary

Dhruva CDM demonstrates that a rigorous astrodynamics engine—incorporating two-body gravity, J_2 perturbations, RK4 numerical integration, and Tsiolkovsky propellant modelling—can be packaged within a service-oriented architecture that delivers real-time conjunction risk assessment and autonomous maneuver scheduling to a modern operator interface. The dual-mode conjunction pipeline (spatial grid for streaming, KD-tree for batch evaluation) provides a practical engineering trade-off between UI responsiveness and throughput, sustaining live updates for fleets exceeding 50 satellites against 20+ tracked debris objects without degrading frame rate or warning latency.

The WebSocket streaming architecture eliminates the stale-update and synchronisation pathologies inherent in polling-based designs, ensuring that SAFE/WARNING/CRITICAL state transitions are reflected in the operator display within a single simulation tick. The comprehensive REST API surface enables integration with external mission control systems, rehearsal environments, and automated test harnesses with minimal adapter work. The Orbital Insight frontend—spanning the 2D Mercator ground-track map, 3D Cesium globe renderer, Conjunction Bullseye radar plot, Fleet Fuel & Δv analytics panel, Maneuver Timeline Gantt scheduler, Graveyard Orbit Monitor, and CDM Alerts feed—delivers the full situational awareness surface required by Flight Dynamics Officers operating multi-satellite constellations under real-time conjunction pressure.

XI.B Future Enhancements

Several extensions are identified as high-value next steps for the Dhruva CDM platform.

Higher-fidelity propagation. Incorporating higher-order zonal harmonics (J_3 – J_6), atmospheric drag (NRLMSISE-00), and solar radiation pressure would extend accurate propagation windows from hours to days and improve conjunction probability estimates for VLEO objects.

Probabilistic collision avoidance. Replacing threshold-based risk with a full collision probability (P_c) computation via the Foster/Alfano 2D covariance projection method would align labelling with the $P_c > 10^{-4}$ threshold used by NASA CARA and ESA SST.

Reinforcement learning maneuver planner. A policy trained on simulated conjunction scenarios could optimise multi-burn sequences minimising $|\Delta \vec{v}|$, maximising station-keeping uptime, and respecting thermal cooldown constraints.

Live TLE ingestion and SGP4 ensemble. Integrating a live NORAD TLE feed (CelesTrak / Space-Track.org) with automated SGP4 seeding would allow Dhruva CDM to operate on real orbital populations.

CCSDS CDM standard and inter-agency coordination. Extending the API to the CCSDS CDM message standard would make Dhruva CDM interoperable with other agencies, positioning it as a node in a federated STM architecture.

Onboard autonomy migration. Packaging the evasion planner as a radiation-tolerant flight-software module with deterministic worst-case execution time guarantees would enable fully autonomous onboard avoidance for satellites in prolonged blackout.

Supplementary Artefacts

The following project artefacts are publicly accessible. All links are clickable.

- **Source Repository:** github.com/jeevankumar-m/project-dhruva
Full backend (FastAPI), frontend (Next.js/React), physics engine, Docker configuration, and dataset files.
 - **Demo Video:** <https://youtu.be/fs9eEaVNgIA>
End-to-end walkthrough of the Orbital Insight dashboard: ground-track map, 3D Cesium view, Conjunction Bullseye, autonomous evasion, and burn-log reporting.
-

References

- [1] D. A. Vallado, *Fundamentals of Astrodynamics and Applications*, 4th ed. Microcosm Press, 2013.
- [2] F. R. Hoots and R. L. Roehrich, “Models for Propagation of NORAD Element Sets,” *Spacetrack Report No. 3*, 1980.
- [3] S. Alfano, “Satellite Conjunction Monte Carlo Analysis,” *Advances in the Astronautical Sciences*, AAS 09-233, 2009.
- [4] R. P. Patera, “Satellite Collision Probability for Nonlinear Relative Motion,” *Journal of Guidance, Control, and Dynamics*, 2003. [PDF]
- [5] NASA Office of Safety and Mission Assurance, “Conjunction Assessment Risk Analysis for Spacecraft Operations,” NASA/TM-2007-214751, 2007. [PDF]
- [6] NASA Office of Safety and Mission Assurance, “Debris Avoidance Maneuver Planning Process,” NPR 8715.6A, 2017.
- [7] L. L. Contant-Jorgenson, P. Lála, and K. Schrogl, “Space Traffic Management in the New Space Era,” *arXiv preprint*, arXiv:2101.11886, 2021. [PDF]
- [8] C. Pardini and L. Anselmo, “Space Debris Environment and Orbital Decay Modelling,” *Celestial Mechanics and Dynamical Astronomy*, 2020. [DOI: 10.1007/s10569-020-09975-2]
- [9] Korea Aerospace Research Institute (KARI), “KARISMA: Korea Aerospace Research Institute Space Object Management System,” *Advances in Space Research*, 2019. Autonomous conjunction screening and CDM generation for Korean LEO assets.
- [10] X. Gao, H. Baoyin, and J. Li, “Autonomous Satellite Constellation Control Using Reinforcement Learning,” *arXiv preprint*, arXiv:2009.04050, 2020. [PDF]
- [11] K. E. Tsiolkovsky, “The Exploration of Cosmic Space by Means of Reaction Devices,” *Scientific Review*, 1903.
- [12] FastAPI Documentation. fastapi.tiangolo.com, 2024.
- [13] CesiumJS Documentation. cesium.com/learn/cesiumjs, 2024.